SMARTMOVE

Intelligent Transportation Ecosystem — Chennai

Technology Stack & Architecture Reference

This document explains every technology, library, and external service used to build the SmartMove hackathon prototype, and — for each one — **why it was chosen and what it is responsible for** in the running application. It is organized by layer: frontend, backend, external data/APIs, and the internal architecture patterns that tie them together.

SmartMove compares private transport (car, bike, cycle, walking) against public transport (Chennai buses) for a given trip, scoring each option on travel time, live traffic, estimated emissions, and community-reported safety, then explains the recommendation in plain language.

1. Frontend Stack

Everything the user directly interacts with: the map-first interface, search panel, mode comparison cards, and live navigation view.

Technology	Category	What it's used for
React 19	UI framework	Builds the entire interface as reusable components (search panel, mode cards, bus cards, map overlays). Chosen for fast iteration during a hackathon and huge ecosystem support.
TypeScript	Language	Adds static types across the whole app (API responses, component props, scoring data) so mismatches between frontend and backend are caught at compile time, not at runtime in a demo.
Vite	Build tool / dev server	Serves the app in development with instant hot-reload and bundles it for production. Also proxies `/api` calls to the Express backend during development.
Tailwind CSS v4	Styling	Utility-class styling used to build the clean, professional dashboard look (white background, subtle borders, one blue accent color) quickly without writing custom CSS files per component.
React-Leaflet	Map bindings	React component wrapper around Leaflet.js. Renders the map, route polylines, and all markers (vehicle icon, destination, parking, incidents) declaratively as React state changes.
Leaflet.js	Map engine	The actual interactive map renderer (pan, zoom, tile loading, marker placement). Open-source and free, avoiding any dependency on the Google Maps JS SDK per the project's API constraints.
Browser Geolocation API	Native browser API	`navigator.geolocation.watchPosition()` continuously tracks the user's real GPS position so the vehicle icon moves live during "Start Trip" navigation mode, instead of only fetching a one-time location fix.

2. Backend Stack

A Node.js API server that all external calls are proxied through, so third-party API keys (TomTom, Groq) never reach the browser, and so every integration can have a documented fallback.

Technology	Category	What it's used for
Node.js	Runtime	Executes the server-side JavaScript/TypeScript. Chosen for a unified JS/TS codebase across frontend and backend, and native `fetch` support for calling external APIs.
Express 5	Web framework	Defines the REST API routes: `/api/plan`, `/api/incidents`, `/api/crowd`, `/api/places`, `/api/reverse-geocode`. Lightweight and fast to wire up for a hackathon timeline.
TypeScript (ESM)	Language	Same reasoning as the frontend: strict typing on every provider interface (RoutingProvider, TrafficProvider, ParkingProvider, AIProvider, etc.) so swapping an implementation can't silently break the contract other code depends on.
tsx	Dev runtime	Runs TypeScript directly with `watch` mode auto-restarting the server on file changes, without a separate compile step during development.
better-sqlite3	Database driver	Synchronous, embedded SQLite database with zero setup. Stores crowd-sourced incident reports, bus crowd-level reports, and the imported/seeded GTFS public-transport dataset (routes, stops, trips, stop_times).
dotenv	Config loading	Loads API keys (TomTom, Groq, optional OpenRouteService/Foursquare) from a local `.env` file that is never committed to version control.
cors	Middleware	Allows the Vite dev server (a different port) to call the Express API during development.
adm-zip + csv-parse	File parsing	Powers the GTFS import pipeline: unzips an official transit data export and parses its CSV files (routes.txt, stops.txt, trips.txt, stop_times.txt) into the SQLite schema, so a real CUMTA/MTC feed can replace the demo dataset with one command.
groq-sdk	AI client	Official client for calling the Groq inference API, used by the AI recommendation layer (see Section 4).

3. External Data Sources & APIs

The project brief specifically ruled out Google Maps APIs. Every service below is free or has a genuinely usable free tier, verified before integration.

Technology	Category	What it's used for
OpenStreetMap tiles	Map display	The actual map imagery (roads, buildings, place labels) rendered by Leaflet. Free, open, no API key required for reasonable non-commercial use.
Nominatim	Geocoding + autocomplete	Converts place names typed by the user ("VIT Chennai") into coordinates, converts GPS coordinates back into a readable address (reverse geocoding), and powers the live address-suggestion dropdown as the user types. Free, no key, rate-limited to ~1 request/sec with a required User-Agent header — both respected via server-side throttling and caching.
OSRM	Routing engine	Calculates the actual driving route geometry, distance, and baseline duration between two points for the public demo server. Free, no key. A documented limitation: the public demo only truly serves the driving profile, so cycling/walking durations are computed from the routed distance using standard average speeds instead of trusting the (identical) demo-server duration.
TomTom Traffic API	Live traffic	Provides real current-vs-free-flow speed data along the route, used to classify traffic as Low/Moderate/Heavy and compute a live delay added on top of the OSRM baseline time. Free tier: 2,500 non-tile requests/day. Falls back to a clearly-labeled time-of-day heuristic (not claimed as live) if the key is missing or the call fails.
Overpass API	Parking data	Queries OpenStreetMap for tagged parking locations (`amenity=parking`) near the destination. Free, no key. Only ever shows "Parking location" — never a fabricated real-time space count, since OSM doesn't provide that data.
Groq (openai/gpt-oss-20b)	AI recommendation	Generates the one-sentence, plain-language explanation of why a mode is recommended, grounded in the real numbers computed by the scoring engine (it is told the answer and the data, and asked only to explain it — never to invent numbers). Free tier: 14,400 requests/day, no card. Falls back to a deterministic rule-based sentence generator if the API call fails.

Note: Chennai public transport (bus routes/stops/times) uses a hand-built demo GTFS dataset seeded directly into SQLite, clearly labeled "Demo data" in the UI, since no current official CUMTA/MTC GTFS export was available at build time. A full GTFS-zip importer (`npm run import-gtfs`) is ready to ingest an official feed the moment one is provided.

4. Internal Architecture & Logic

Beyond off-the-shelf libraries, several purpose-built systems make the app more than a route lookup tool:

Technology	Category	What it's used for
Provider interfaces	Design pattern	Every external integration (routing, traffic, parking, public transport, AI) is defined as a TypeScript interface with at least one real implementation and one fallback implementation. This means a provider can be swapped (e.g. a new AI vendor) without touching any calling code.
RouteScoringService	Scoring engine	Combines time, traffic, emissions, and safety into one 0–100 "overall score" using configurable weights. Includes a practicality multiplier so a mode that's drastically slower than the fastest option (e.g. a 10-hour walk) can't out-score a fast option just by having perfect emissions/safety numbers.
EmissionService	Emissions estimator	Calculates estimated CO2 output per mode from distance and a configurable grams-per-km factor table (car, bike, cycle, walk, bus, metro), always labeled as an estimate, never presented as a measured figure.
IncidentService	Crowd-sourced safety	Powers the "Report Incident" (SOS) button: any user can report an accident/hazard location and severity. Other users see it as a warning marker on the map, and it factors into a route's safety score — always labeled as a community report, never as an official statistic.
CrowdService	Bus crowd reporting	Lets riders report how crowded a specific bus is (Low/Moderate/High). Aggregates recent reports with a recency-weighted average, so a report from 5 minutes ago counts more than one from 80 minutes ago.
GTFS import pipeline	Transit data ingestion	Parses standard GTFS zip files into the SQLite schema and answers "which buses connect these two points" by matching stops within a radius of the origin/destination and walking the stop_times table for matching trips.